



ARTIFICIAL NEURAL NETWORKS (ANN) LAB

11

Ms. Shakeela Shaheen

LAB 11: USING LSTM FOR TIME SERIES DATA FORECASTING

Learning Objectives

By the end of this lab, students will:

- Understand **time series data**
- Understand **trend & seasonality**
- Prepare sequential data for forecasting
- Implement **LSTM for regression (not classification)**
- Predict **future values based on past sequences**
- Analyze model performance
- Understand how LSTM captures **temporal dependencies**

PART 1 – Motivation (Why Time Series?)

Many real-world problems depend on time:

- Stock prices
- Weather
- Sales forecasting
- Energy consumption

Normal Data	Time Series
Independent samples	Dependent samples
Order doesn't matter	Order is critical
No memory needed	Memory required

In time series:

x_t depends on $x_{t-1}, x_{t-2}, \dots$

Future depends on the past **(Unlike Markov models that rely only on recent states, LSTMs can learn long-term dependencies in time series data.)**

Example:

[100, 120, 130, 150, 170] → ?

We want to predict:

Next value = future

Why MLP Fails Here?

MLP sees:

$x = [100, 120, 130, 150]$

But:

- No sense of time
- No order awareness
- No memory

Why LSTM?

LSTM:

- Remembers past values
- Learns patterns (trend + seasonality)
- Handles long-term dependencies

PART 2 – Components of Time Series

Understanding this is **VERY important before modeling**

Trend:

Long-term increase or decrease

Example:

- Sales growing every year

Seasonality:

Repeating patterns at fixed intervals

Example:

- Airline passengers increase every summer

- More shopping in December

Noise:

Random fluctuations

Example:

- Sudden spike due to unexpected event

Visualization Intuition

$$\text{Time series} = \text{Trend} + \text{Seasonality} + \text{Noise}$$

PART 3 – Types of Time Series Problems**1. Forecasting:** Predict future values

Example:

[100, 120, 130, 150] → ?

2. Classification

Predict category based on sequence

3. Anomaly Detection

Detect unusual spikes

PART 4 – Stationarity (VERY IMPORTANT)**What is Stationarity?**

A time series is **stationary** if:

- Mean is constant
- Variance is constant
- No trend over time

Non-Stationary Example:

- Increasing sales every year

Why it matters?

Many models fail if data is non-stationary

PART 5 – Traditional vs Deep Learning Approach

Traditional Models (ARIMA etc.)

- Require stationarity
- Manual feature engineering
- Limited for complex patterns

LSTM Approach

- Learns patterns automatically
- Handles non-linear relationships
- Captures long-term dependencies

PART 6 – How LSTM Solves Time Series

Recall from Lab 08:

LSTM uses:

- Memory cell C_t
- Hidden state h_t
- Gates (forget, input, output)

Key Idea in Time Series

Instead of predicting directly

We convert time series into: **Supervised learning problem**

Sliding Window Concept (VERY IMPORTANT)

This is **core to LSTM time series forecasting**

Intuition (Super Simple):

Time series is like a story:

To predict tomorrow → look at the last few days

That “last few days” = **window**

Example Dataset:

[10, 20, 30, 40, 50]

Window size = 3

We convert into:

Input (X)	Output (y)
[10, 20, 30]	40
[20, 30, 40]	50

This is called **supervised learning transformation**

Why do we do this?

Because LSTM **does NOT understand raw sequences directly** like:

10, 20, 30, 40, 50

Instead, it learns patterns like:

“If last 3 values are X → next value is Y”

Visual Understanding

Time → →

[10, 20, 30] → 40

[20, 30, 40] → 50

Window “slides” forward → hence name **Sliding Window**

Window Size (VERY IMPORTANT)

- Small window → misses long-term patterns
- Large window → more context but harder to train

Typical values:

- 3–10 (simple datasets)
- 30+ (complex time series)

Why this works?

LSTM learns:

$$f(x_{t-3}, x_{t-2}, x_{t-1}) \rightarrow x_t$$

PART 7 – Mathematical View

At time step t :

$$h_t = LSTM(x_t, h_{t-1}, C_{t-1})$$
$$y_t = W \cdot h_t$$

Output depends on:

- Current input
- Past memory

PART 8 – Practical Implementation

Load Dataset:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

df = pd.read_csv('AirPassengers.csv')
```

```
plt.plot(df['#Passengers'])
plt.title("Time Series Data")
plt.show()
```

```
from sklearn.preprocessing import MinMaxScaler

scaler = MinMaxScaler()
data_scaled = scaler.fit_transform(df[['#Passengers']])
```

Create Sequences:

```
def create_sequences(data, window_size):
    X, y = [], []
    for i in range(len(data) - window_size):
        X.append(data[i:i+window_size])
        y.append(data[i+window_size])
    return np.array(X), np.array(y)

window_size = 12
X, y = create_sequences(data_scaled, window_size)
```

Train/Test Split and Building LSTM:

```
split = int(0.8 * len(X))

X_train, X_test = X[:split], X[split:]
y_train, y_test = y[:split], y[split:]
```

```
import tensorflow as tf

model = tf.keras.Sequential([
    tf.keras.layers.LSTM(50, activation='tanh', input_shape=(window_size,1)),
    tf.keras.layers.Dense(1)
])
```

```
model.compile(optimizer='adam', loss='mse')
```

```
history = model.fit(
    X_train, y_train,
    epochs=20,
    validation_data=(X_test, y_test)
)
```

```
predictions = model.predict(X_test)

predictions = scaler.inverse_transform(predictions)
y_test_actual = scaler.inverse_transform(y_test)
```

```
plt.plot(y_test_actual, label='Actual')
plt.plot(predictions, label='Predicted')
plt.legend()
plt.show()
```


LAB TASK

Scenario

You are working as an **AI Engineer at a retail company**.

The company wants to:

 Forecast **monthly product demand**

Problem:

- Demand fluctuates seasonally
- Incorrect prediction → losses

Dataset

You can use:

- Walmart Store Sales Forecasting Dataset

Tasks

Task 1 – Data Understanding

- Plot time series
- Identify:
 - Trend
 - Seasonality

Task 2 – Preprocessing

- Normalize data
- Create sequences (window size = 12)

Task 3 – LSTM Model

- Build LSTM model
- Train for 15–20 epochs

Task 4 – Parameter Modification (CLO 2)

Modify ANY TWO:

- Window size (6, 12, 24)
- LSTM units (20, 100)

- Add Dropout

Compare:

Model	Train Loss	Val Loss
Base		
Modified		

Task 5 – Forecast Future

- Predict next 12 time steps
- Plot forecast

Required Plots

- Actual vs Predicted
- Loss vs Epochs
- Future forecast